Node.js + FFmpeg Video Processing Pipeline

A practical backend project for process management, media transcoding, queues, and streaming delivery

Project outcome

Upload a video, process it in a background worker, generate multiple video outputs, expose job progress, and serve streamable HLS playback from an Express API.

Upload API **FFmpeg Worker** **HLS Streaming** **Live Progress** **Metadata + Thumbnails**

Backend	Processing	Queue	Storage
Express + Multer	FFmpeg + ffprobe	BullMQ + Redis	Local disk first

1. Project Overview

Build a mini Cloudinary or YouTube-style video processing service. The goal is not only to run FFmpeg from Node.js, but to design a realistic backend workflow around long-running media jobs.

> **Final milestone:** Upload one .mp4 file, watch the job progress update live, then play the generated HLS stream in a browser.

Output	Purpose
input.mp4	The original uploaded file stored safely for processing.
360p.mp4 / 720p.mp4	Transcoded MP4 renditions for different device and network conditions.
master.m3u8 + segments	HLS stream output for browser playback using hls.js.
thumbnail.jpg	Poster image from a selected timestamp.
animated-preview.gif	Short animated preview for hover cards or listing pages.
metadata.json	Duration, resolution, codecs, bitrate, and other ffprobe data.

2. Architecture

Keep video processing out of the HTTP request lifecycle. The API should create a job quickly, then a separate worker should perform the CPU-heavy FFmpeg work.

```
Client
  |
  v
Express API  -- stores upload --> uploads/{videoId}/original.mp4
  |
  | creates job
  v
BullMQ / Redis Queue
  |
  v
Video Worker
  |-- ffprobe metadata
  |-- thumbnail generation
  |-- MP4 transcoding
  |-- HLS generation
  |-- progress updates
  v
outputs/{videoId}/...
  |
  v
Express static routes / streaming endpoints
```

Component	Responsibility
Express API	Accepts uploads, returns job IDs, serves status and

	generated assets.
Queue	Decouples request handling from long-running FFmpeg jobs.
Worker	Runs FFmpeg and ffprobe processes, updates progress, records failures.
Storage service	Owns paths, output folders, cleanup, and static asset URLs.
Job database	Stores status, progress, current step, errors, and metadata.

3. API Design

Method	Route	Description
POST	/videos	Upload a video and create a queued processing job.
GET	/videos/:id/status	Return job status, current processing step, progress, and errors.
GET	/videos/:id/metadata	Return ffprobe metadata and generated asset information.
GET	/videos/:id/thumbnail	Serve the poster image.
GET	/videos/:id/preview	Serve animated preview or short MP4 preview.
GET	/videos/:id/stream	Return or redirect to the HLS master playlist.
DELETE	/videos/:id	Cancel or delete a video job and remove generated files.

Upload response

```
{
  "videoId": "vid_123",
  "status": "queued"
}
```

Status response

```
{
  "videoId": "vid_123",
  "status": "processing",
  "progress": 64,
  "currentStep": "generating_hls"
}
```

4. Processing Workflow

Step	What happens
queued	Job is created and waiting for an available worker.
probing	Run ffprobe to collect duration, streams, resolution, codec, bitrate, and frame rate.

thumbnailing	Generate poster image and optional sprite thumbnails.
transcoding_mp4	Create MP4 renditions such as 360p and 720p.
generating_hls	Create HLS playlist and segments for browser streaming.
finalizing	Write metadata, verify output files, and update URLs.
completed / failed	Mark terminal state with result or failure reason.

5. Core FFmpeg Tasks

Wrap each command in a reusable Node.js helper built around child_process.spawn. Use spawn rather than exec so long commands, streaming logs, and large stderr output are handled safely.

Extract metadata with ffprobe

```
ffprobe -v quiet -print_format json -show_format -show_streams input.mp4
```

Generate a thumbnail

```
ffmpeg -ss 00:00:05 -i input.mp4 -vframes 1 -q:v 2 thumbnail.jpg
```

Create a 360p MP4

```
ffmpeg -i input.mp4 \
 -vf scale=-2:360 \
 -c:v libx264 -preset fast -crf 23 \
 -c:a aac output-360p.mp4
```

Create a 720p MP4

```
ffmpeg -i input.mp4 \
 -vf scale=-2:720 \
 -c:v libx264 -preset fast -crf 23 \
 -c:a aac output-720p.mp4
```

Generate HLS output

```
ffmpeg -i input.mp4 \
 -profile:v baseline -level 3.0 \
 -start_number 0 -hls_time 6 -hls_list_size 0 \
 -f hls output/index.m3u8
```

Generate animated preview GIF

```
ffmpeg -ss 00:00:05 -t 4 -i input.mp4 \
 -vf fps=10,scale=480:-1:flags=lanczos preview.gif
```

6. Node.js Process Helper

```
const { spawn } = require("child_process");

function runProcess(command, args, options = {}) {
 return new Promise((resolve, reject) => {
  const child = spawn(command, args, { stdio: ["ignore", "pipe", "pipe"] });
  let stderr = "";
```

```
    child.stderr.on("data", chunk => {
      const text = chunk.toString();
      stderr += text;
      options.onLog?.(text);
    });

    child.on("error", reject);

    child.on("close", code => {
      if (code === 0) resolve({ code });
      else reject(new Error(`${command} exited with code ${code}
${stderr}`));
    });
  });
}

module.exports = { runProcess };
```

7. Suggested Folder Structure

```
video-pipeline/
├── src/
│   ├── server.js
│   ├── routes/
│   │   └── videos.js
│   ├── services/
│   │   ├── ffmpeg.js
│   │   ├── ffprobe.js
│   │   ├── storage.js
│   │   └── progress.js
│   ├── queues/
│   │   ├── videoQueue.js
│   │   └── videoWorker.js
│   └── db/
│       └── jobs.js
├── uploads/
├── outputs/
├── package.json
└── README.md
```

8. Implementation Plan

Phase	Goal
Phase 1 - CLI proof of concept	Process one local video and generate thumbnail, preview, and metadata.
Phase 2 - Express upload API	Accept uploads with Multer, save files by videoId,

	return queued job response.
Phase 3 - Worker and queue	Add BullMQ + Redis, process videos outside the request lifecycle.
Phase 4 - Multiple outputs	Generate MP4 renditions, HLS playlist, thumbnail, preview, and metadata JSON.
Phase 5 - Progress and errors	Expose /status, parse FFmpeg logs, persist failed state and error messages.
Phase 6 - Frontend playback	Add upload UI, progress updates, and HLS playback with hls.js.

9. Extra Challenge Features

- Watermark videos with a logo overlay.
- Generate sprite thumbnails for video scrubbing.
- Extract audio as MP3.
- Detect black frames, silent audio, or very low-resolution uploads.
- Retry failed jobs with exponential backoff.
- Cancel a running FFmpeg process from an API endpoint.
- Clean up old uploads and outputs with a scheduled job.
- Store job records in SQLite or Postgres.
- Dockerize the API, Redis, worker, and FFmpeg runtime.
- Add a small React frontend with upload progress and playback.

10. Testing Checklist

Area	Test
Upload validation	Reject missing files, unsupported extensions, empty files, and oversized files.
Happy path	Upload a valid MP4 and assert all expected output files exist.
Failure path	Upload a corrupted video and verify the job becomes failed with a useful reason.
Progress	Confirm status moves through queued, probing, transcoding, HLS, and completed.
Streaming	Open the generated HLS playlist in a browser player.
Cleanup	Delete a job and verify its uploads and outputs are removed.

11. Success Criteria

- The HTTP request returns quickly after creating a processing job.
- FFmpeg runs in a background worker, not inside the upload request handler.
- The service generates metadata, thumbnail, MP4 renditions, and HLS output.
- The status endpoint reports progress and useful error states.
- The browser can play the generated HLS stream.
- The code separates API routes, queue logic, storage paths, FFmpeg helpers, and job persistence.